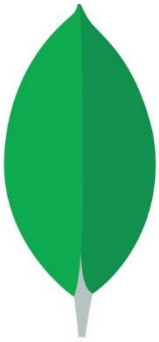
SBD

Actividad 6.4

PROYECTO
KAFKA + MONGODB



mongoDB®

1. Levantar Docker y crear el topic

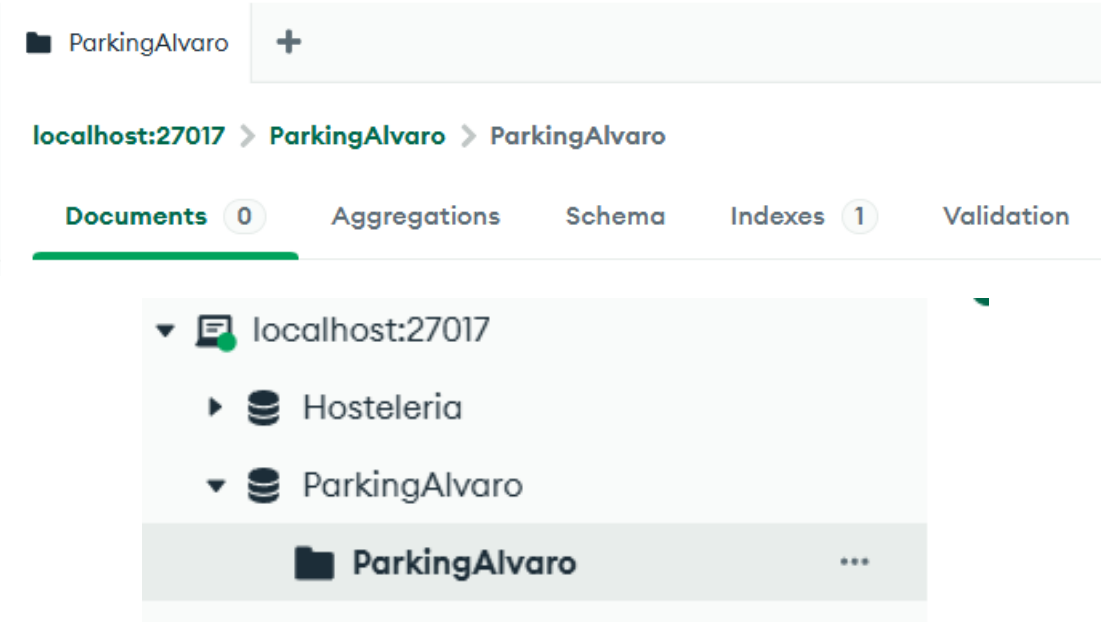
```
docker exec -it kafka kafka-topics --create --topic ParkingAlvaro --bootstrap-server localhost:9092 --partitions 1 --replication-factor 1
```

- Primero levantamos Docker para crear el topic con el nombre que se pide en la actividad, y listamos para comprobar que se haya creado:

```
Windows PowerShell
PS C:\Users\alvar\Desktop\Nueva carpeta\IA Y BIG DATA\SBD\UT6_6.4> docker compose up -d
[+] up 2/2
 ✔Container kafka      Running
                            0.0s
 ✔Container zookeeper Running
                            0.0s
PS C:\Users\alvar\Desktop\Nueva carpeta\IA Y BIG DATA\SBD\UT6_6.4> docker exec -it kafka
kafka-topics --create --topic ParkingAlvaro --bootstrap-server localhost:9092 --partition
s 1 --replication-factor 1
Created topic ParkingAlvaro.
PS C:\Users\alvar\Desktop\Nueva carpeta\IA Y BIG DATA\SBD\UT6_6.4>
PS C:\Users\alvar\Desktop\Nueva carpeta\IA Y BIG DATA\SBD\UT6_6.4> docker exec -it kafka
kafka-topics --list --bootstrap-server localhost:9092
ParkingAlvaro
PS C:\Users\alvar\Desktop\Nueva carpeta\IA Y BIG DATA\SBD\UT6_6.4>
```

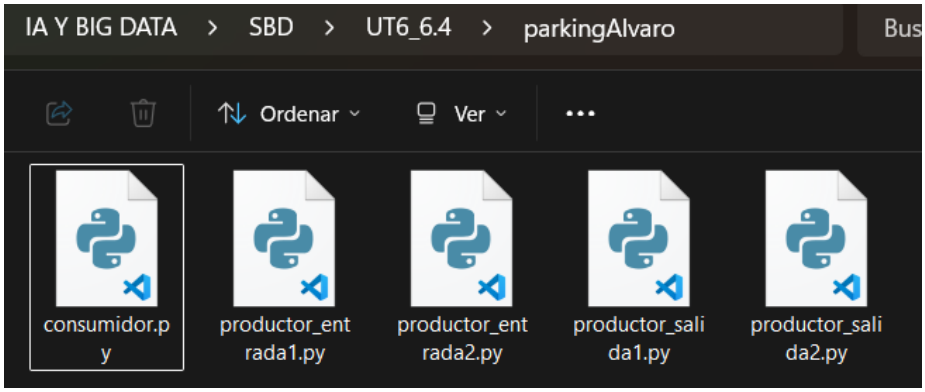
2. Arrancar el consumidor

- Simplemente creamos la base de datos y la colección con el nombre requerido desde Compass



3. Crear estructura y archivos

- Voy a crear una carpeta para el proyecto donde irán los archivos necesarios (el consumidor y los 4 productores).



- Todos los productores tienen un código casi idéntico, se empieza importando las librerías necesarias, definimos variables con las **matriculas** y el **topic**, y definimos el producer. Código común:

```
from kafka import KafkaProducer
import json
import time
import random

# Lista fija de matrículas
matriculas = [
    "12839KLM", "45092TRZ", "94830QWE", "73821XCV", "10923BNM",
    "34567ZXC", "78234QAZ", "39847EDC", "56123RFV", "78401TGB",
    "93218YHN", "87245UJM", "10029IKL", "45903OLP", "88923LKJ",
    "23789MNB", "12048POI", "98347UYT", "47821TRE", "37492WSX",
    "94567EDV", "68345GBH", "15423NHY", "34987UJH", "22019IKM",
    "32048PLK", "40298MNJ", "82930XCD", "74923DFR", "98234WSA",
    "73482JHG", "15390REW", "98234TGY", "40001ZXV", "38201MLK",
    "68492PNM", "38219UIK", "52348OPA", "93484GHK", "76420LUI",
    "62190KJU", "89012TRE", "19384WSD", "80239CDE", "54093LOI",
    "74821JLO", "92384MKL", "28493BGT", "67492NAQ", "58129OMI"
]

# Nombre del tppic de Kafka
topic = "ParkingAlvaro"

# Creamos el productor de Kafka.
# bootstrap_servers: dónde está Kafka.
# value_serializer: transforma el diccionario Python en JSON antes de
enviarlo.
producer = KafkaProducer(
    bootstrap_servers="localhost:9092",
    value_serializer=lambda v: json.dumps(v).encode("utf-8")
)
```

a) **productor_entrada1.py** y **productor_entrada2.py**

```
# Bucle infinito para que el productor envíe mensajes
cada 10 segundos
while True:
    matricula = random.choice(matriculas)

    mensaje = {
        "lector": "ENTRADA1",
        "tipo": "entrada",
        "matricula": matricula
    }

    producer.send(topic, value=mensaje)
    producer.flush()

    print("Mensaje enviado:", mensaje)
    print("\n- Mensaje enviado por el productor de
Entrada1")

    time.sleep(10)
```

```
while True:
    matricula = random.choice(matriculas)

    mensaje = {
        "lector": "ENTRADA2",
        "tipo": "entrada",
        "matricula": matricula
    }

    producer.send(topic, value=mensaje)
    producer.flush()

    print("Mensaje enviado:", mensaje)
     print("\n- Mensaje enviado por el productor de
Entrada2")
    time.sleep(10)
```

b) **productor_salida1** y **productor_salida2**

```
while True:
    matricula = random.choice(matriculas)

    mensaje = {
        "lector": "SALIDA1",
        "tipo": "salida",
        "matricula": matricula
    }

    producer.send(topic, value=mensaje)
    producer.flush()

    print("Mensaje enviado:", mensaje)
    print("\n- Mensaje enviado por el productor de SALIDA1\n")

    time.sleep(10)
```

```
while True:
    matricula = random.choice(matriculas)

    mensaje = {
        "lector": "SALIDA2",
        "tipo": "salida",
        "matricula": matricula
    }

    producer.send(topic, value=mensaje)
    producer.flush()

    print("Mensaje enviado:", mensaje)
    print("\n- Mensaje enviado por el productor de SALIDA2\n")

    time.sleep(10)
```

- Con esto tendríamos 4 productores, 2 de entrada y 2 de salida, donde cada uno se conecta a Kafka, envía mensajes al topic *ParkingAlvaro* y cada 10 segundos selecciona de forma aleatoria una matrícula del array proporcionado.
- El mensaje incluye 3 campos: el código del lector, el tipo de lectura (entrada/salida) y la matrícula.

c) consumidor.py

```python
from kafka import KafkaConsumer
from pymongo import MongoClient
import json

# Topic obligatorio con el nombre del alumno.
topic = "ParkingAlvaro"

# Conectamos con Kafka y leemos mensajes desde el principio del topic.
# value_deserializer convierte los mensajes JSON de Kafka en diccionarios
Python.
consumer = KafkaConsumer(
    topic,
    bootstrap_servers="localhost:9092",
    auto_offset_reset="earliest",
    enable_auto_commit=True,
    group_id="grupo_parking_alvaro",
    value_deserializer=lambda v: json.loads(v.decode("utf-8"))
)

# Conectamos con MongoDB local.
# La base de datos y la colección deben llamarse ParkingAlvaro.
client = MongoClient("mongodb://localhost:27017")
db = client["ParkingAlvaro"]
collection = db["ParkingAlvaro"]

print("Consumidor iniciado. Esperando mensajes...")

# Bucle infinito para ir leyendo mensajes del topic y guardarlos en
MongoDB.
for mensaje in consumer:
    dato = mensaje.value
    collection.insert_one(dato)
    print("Insertado en MongoDB:", dato)
```

4. Lanzar productores y consumidor:

- Productores:

- Productor Entrada 1:

```
Consumidor Alvaro   ProdEntrada1   ProdEntrada2   ProdSalida1   ProdSalida2

PS C:\Users\alvar\Desktop\Nueva carpeta\IA Y BIG DATA\SBD\UT6_6.4\parkingAlvaro> python .\prod
Mensaje enviado: {'lector': 'ENTRADA1', 'tipo': 'entrada', 'matricula': '73821XCV'}

- Mensaje enviado por el productor de Entrada1

Mensaje enviado: {'lector': 'ENTRADA1', 'tipo': 'entrada', 'matricula': '47821TRE'}

- Mensaje enviado por el productor de Entrada1
```

- Productor Entrada 2:

```
Consumidor Alvaro | ProdEntrada1 | ProdEntrada2 | ProdSalida1 | ProdSalida2
PS C:\Users\alvar\Desktop\Nueva carpeta\IA Y BIG DATA\SBD\UT6_6.4\parkingAlvaro> python .\productor_entrada2.py
Mensaje enviado: {'lector': 'ENTRADA2', 'tipo': 'entrada', 'matricula': '23789MNB'}

- Mensaje enviado por el productor de Entrada2

Mensaje enviado: {'lector': 'ENTRADA2', 'tipo': 'entrada', 'matricula': '52348OPA'}

- Mensaje enviado por el productor de Entrada2
```

- Productor Salida 1:

```
Consumidor Alvaro | ProdEntrada1 | ProdEntrada2 | ProdSalida1 | ProdSalida2
PS C:\Users\alvar\Desktop\Nueva carpeta\IA Y BIG DATA\SBD\UT6_6.4\parkingAlvaro> python .\productor_salida1.py
Mensaje enviado: {'lector': 'SALIDA1', 'tipo': 'salida', 'matricula': '54093LOI'}

- Mensaje enviado por el productor de SALIDA1

Mensaje enviado: {'lector': 'SALIDA1', 'tipo': 'salida', 'matricula': '15390REW'}

- Mensaje enviado por el productor de SALIDA1
```

- Productor Salida 2:

```
Consumidor Alvaro | ProdEntrada1 | ProdEntrada2 | ProdSalida1 | ProdSalida2
PS C:\Users\alvar\Desktop\Nueva carpeta\IA Y BIG DATA\SBD\UT6_6.4\parkingAlvaro> python .\productor_salida2.py
Mensaje enviado: {'lector': 'SALIDA2', 'tipo': 'salida', 'matricula': '98347UYT'}

- Mensaje enviado por el productor de SALIDA2

Mensaje enviado: {'lector': 'SALIDA2', 'tipo': 'salida', 'matricula': '12048POI'}

- Mensaje enviado por el productor de SALIDA2
```

- **Consumidor** (Antes de arrancar los productores):

```
Consumidor Alvaro
PS C:\Users\alvar\Desktop\Nueva carpeta\IA Y BIG DATA\SBD\UT6_6.4\parkingAlvaro> python .\consumidor.py
Consumidor iniciado. Esperando mensajes...
```

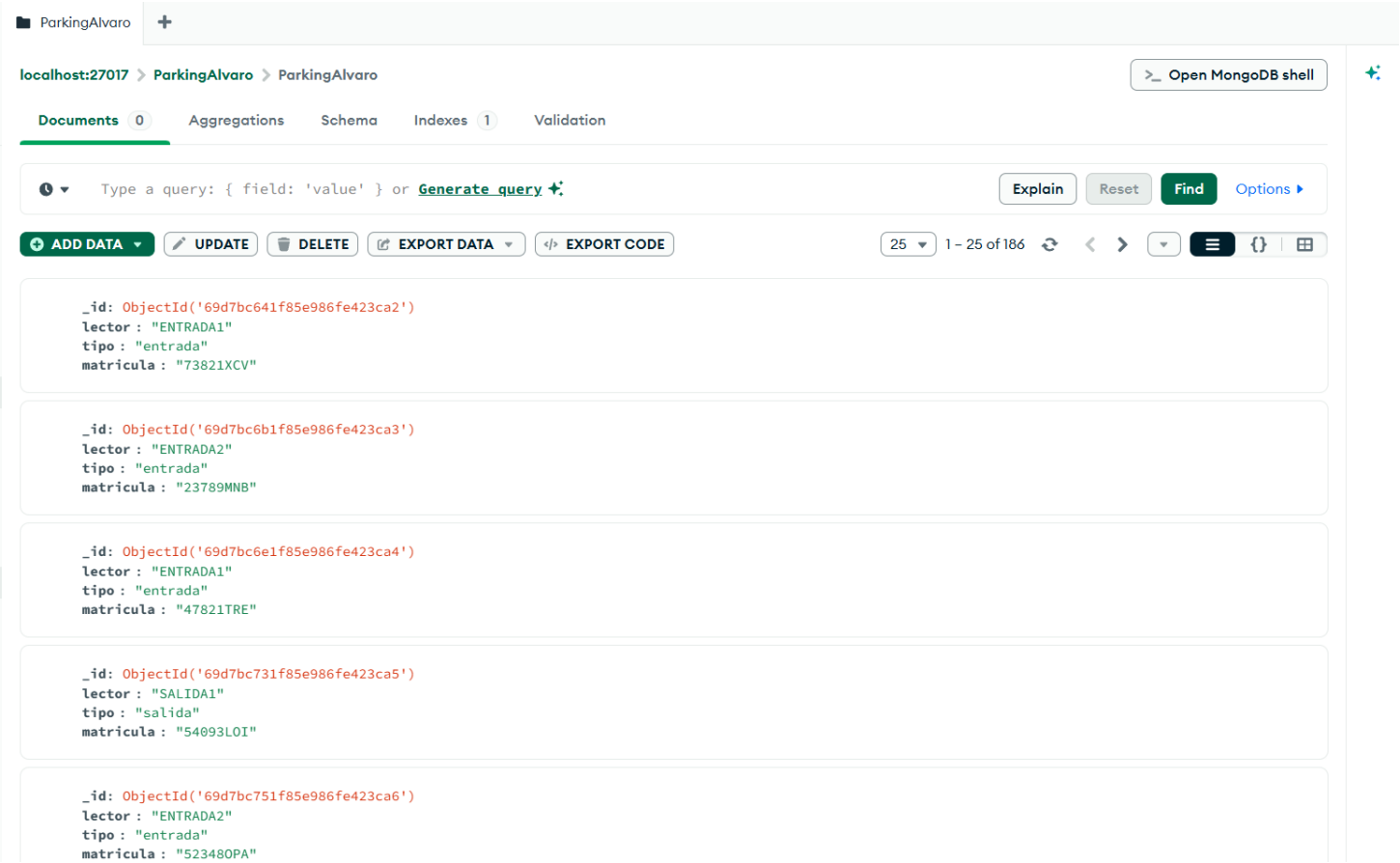
- **Consumidor** (Productores arrancados):

```
Consumidor Alvarc | ProdEntrada1 | ProdEntrada2 | ProdSalida1 | ProdSalida2
PS C:\Users\alvar\Desktop\Nueva carpeta\IA Y BIG DATA\SBD\UT6_6.4\parkingAlvaro> python .\consumidor.py
Consumidor iniciado. Esperando mensajes...
Insertado en MongoDB: {'lector': 'ENTRADA1', 'tipo': 'entrada', 'matricula': '73821XCV', '_id': ObjectId('69d7bc641f8
5e986fe423ca2')}
Insertado en MongoDB: {'lector': 'ENTRADA2', 'tipo': 'entrada', 'matricula': '23789MNB', '_id': ObjectId('69d7bc6b1f8
5e986fe423ca3')}
Insertado en MongoDB: {'lector': 'ENTRADA1', 'tipo': 'entrada', 'matricula': '47821TRE', '_id': ObjectId('69d7bc6e1f8
5e986fe423ca4')}
Insertado en MongoDB: {'lector': 'SALIDA1', 'tipo': 'salida', 'matricula': '54093LOI', '_id': ObjectId('69d7bc731f85e
986fe423ca5')}
```

- Podemos ver que los productores mandan el mensaje correspondiente y que el consumidor recibe cada mensaje y lo sube a la base de datos.

5. Base de datos MongoDB

- Después de 10 minutos con el consumidor y los productores arrancados los vamos a cerrar y comprobar que se han subido los registros a la base de datos:



6. Análisis de los datos

- Como análisis principal voy a crear una consulta en **Compass**.

La consulta agrupa todos los documentos por matrícula. Después cuenta cuántas lecturas de entrada y cuántas lecturas de salida tiene cada matrícula. A continuación, añade un campo llamado estado para clasificar cada matrícula como **correcta_salio, correcta_sigue_dentro o incorrecta**, según la relación entre el número de entradas y salidas. Finalmente, añade un campo errores con la diferencia absoluta entre entradas y salidas para medir el nivel de incoherencia de cada matrícula.

```
[
  {
    "$group": {
      "_id": "$matricula",
      "entradas": {
        "$sum": {
          "$cond": [
            { "$eq": ["$tipo", "entrada"] },
```

```
            1,
            0
          ]
        }
      },
      "salidas": {
        "$sum": {
          "$cond": [
            { "$eq": ["$tipo", "salida"] },
            1,
            0
          ]
        }
      }
}
  },
  {
    "$addFields": {
      "estado": {
        "$switch": {
          "branches": [
            {
              "case": { "$eq": ["$entradas", "$salidas"] },
              "then": "correcta_salio"
            },
            {
              "case": { "$eq": ["$entradas", { "$add": ["$salidas", 1] }]
},
              "then": "correcta_sigue_dentro"
            }
          ],
          "default": "incorrecta"
        }
      },
      "errores": {
        "$abs": {
          "$subtract": ["$entradas", "$salidas"]
        }
      }
    }
  }
]
```

- Muestra general de 3 documentos consultados:

PIPELINE OUTPUT PREVIEW
Sample of 10 documents
OUTPUT OPTIONS

```
_id: "10923BNM"
entradas : 9
salidas : 8
estado : "correcta_sigue_dentro"
errores : 1

_id: "76420LUI"
entradas : 10
salidas : 7
estado : "incorrecta"
errores : 3

_id: "73482JHG"
entradas : 11
salidas : 13
estado : "incorrecta"
errores : 2
```

76420LUI

- Ahora vamos a sacar algunos datos numéricos como las cantidades de matrículas en cada estado:

a) Número de matrículas correctas:

```
[
 {
  "$group": {
   "_id": "$matricula",
   "entradas": {
    "$sum": {
     "$cond": [{ "$eq": ["$tipo", "entrada"] }, 1, 0]
    }
   },
   "salidas": {
    "$sum": {
     "$cond": [{ "$eq": ["$tipo", "salida"] }, 1, 0]
    }
   }
  }
 },
 {
  "$addFields": {
   "correcta": {
    "$or": [
     { "$eq": ["$entradas", "$salidas"] },
     { "$eq": ["$entradas", { "$add": ["$salidas", 1] }] }
    ]
   }
  }
 },
 {
  "$match": {
   "correcta": true
  }
 },
 {
  "$count": "matriculas_correctas"
 }
```

PIPELINE OUTPUT PREVIEW
Sample of 1 document
OUTPUT OPTIONS

```
matriculas_correctas : 4
```

b) Número de matrículas correctas que siguen en el parking:

```
[
 {
  "$group": {
   "_id": "$matricula",
   "entradas": {
    "$sum": {
     "$cond": [{ "$eq": ["$tipo", "entrada"] }, 1, 0]
    }
   },
   "salidas": {
    "$sum": {
     "$cond": [{ "$eq": ["$tipo", "salida"] }, 1, 0]
    }
   }
  }
 },
 {
  "$match": {
   "$expr": {
    "$eq": ["$entradas", { "$add": ["$salidas", 1] }]
   }
  }
 },
 {
  "$count": "siguen_en_parking"
 }
```

PIPELINE OUTPUT PREVIEW

OUTPUT OPTIONS

Sample of 1 document

```
siguen_en_parking : 2
```

c) Número de matrículas con lecturas incorrectas:

```
 {
  "$group": {
   "_id": "$matricula",
   "entradas": {
    "$sum": {
     "$cond": [{ "$eq": ["$tipo", "entrada"] }, 1, 0]
    }
   },
   "salidas": {
    "$sum": {
     "$cond": [{ "$eq": ["$tipo", "salida"] }, 1, 0]
    }
   }
  }
 },
 {
  "$match": {
   "$expr": {
    "$and": [
     { "$ne": ["$entradas", "$salidas"] },
     { "$ne": ["$entradas", { "$add": ["$salidas", 1] }] }
    ]
   }
  }
 },
 {
  "$count": "matriculas_incorrectas"
}
```

d) Media de matrículas erróneas

```
[
 {
  "$group": {
   "_id": "$matricula",
   "entradas": {
    "$sum": {
     "$cond": [{ "$eq": ["$tipo", "entrada"] }, 1, 0]
    }
   },
   "salidas": {
    "$sum": {
     "$cond": [{ "$eq": ["$tipo", "salida"] }, 1, 0]
    }
  }
 },
 {
  "$addFields": {
   "incorrecta": {
    "$and": [
     { "$ne": ["$entradas", "$salidas"] },
     { "$ne": ["$entradas", { "$add": ["$salidas", 1] }] }
    ]
   },
   "errores": {
    "$abs": {
     "$subtract": ["$entradas", "$salidas"]
    }
   }
 },
 {
  "$match": {
   "incorrecta": true
  }
 },
 {
  "$group": {
   "_id": null,
   "media_errores": { "$avg": "$errores" }
  }
```

7. Conclusion final

- Considero que ha sido una actividad interesante donde lo que más me ha llamado la atención es el cómo varios productores pueden ir generando eventos y mensajes en paralelo y un consumidor vaya almacenando todos esos datos.

  Como mejora, ya se comenta en el enunciado, pero ciertamente añadir campos de fecha y hora ayudaría tanto al análisis como a tener un mejor historial de cada vehículo y detectar errores con más precisión.

  Otra adición podría ser validar el campo de la matrícula para comprobar si es válida.